Blood Donor Eligibility Reminder Console — CA1 Proposal

Nov 5, 2026

Lovely Professional University

School of Computer Applications · Master of Computer Applications (MCA)

CAP455 · Object-Oriented Programming using C++

Minor Project · CA1: Design & Create · Session 2026–27

Field	Details
Student name	Hisham P P
Registration / roll no.	12604325 / 69
Section	D1P2635
Faculty	Adarsh Singh
Allocated project	Blood Donor Eligibility Reminder Console
Submission file name	12604325_CAP455_CA1_BloodDonorReminder

1. Problem statement

Regular blood donors must wait a minimum gap between two whole blood donations, but most blood banks and donation camps do not track when each donor becomes eligible again. Under the NBTC 2017 guidelines this gap is 90 days for male donors and 120 days for female donors.

In practice, donor lists are kept in registers or spreadsheets. When there is a shortage, volunteers call donors who donated recently and are turned away, while donors who are already eligible are never contacted. Donors themselves also forget their last donation date.

The **Blood Donor Eligibility Reminder Console** stores donor records, checks each donor against the basic NBTC eligibility rules, and shows who is eligible to donate today and who becomes eligible in the next 7 days. Because a console program cannot send notifications, the reminder is produced when the program is opened and the user asks for the reminder list.

2. Existing Systems

I worked in this project i understand one thing that is the actually a real problem in india i used to research like whether any app or web done this solution or not ... still now e-RaktKosh is a governments app trying to solve but it still fragmented. My project is a small model of the eligibility logic for donor group's own list, So the group always knows who is eligible today. But i like to work with project more, Like in india there are several patients are getting late to get their blood. I know it's too complicated problem but still I'm researching it towards and let me know if there is any way to try to solve.

3. Objectives and scope

CA1 delivers an in-memory donor register with validated Create and Read operations; storage, inheritance and the full CRUD menu follow in later phases.

Objectives

1. Register donors with complete, validated details and a unique Donor ID.
2. Reject duplicate Donor IDs and invalid input at the point of entry.
3. Display all donors, or one donor searched by ID, with their next eligible date.
4. Show a reminder list: donors eligible today and donors due in the next 7 days.
5. Keep the code readable and modular so later phases can build on it.

In scope for CA1

- Donor entity class, array of objects (maximum 100 donors), pointers, `this`, `std::string`
- Create (add donor) and Read (display all, search by ID, reminder list, find by blood group)
- Input validation for every field; no duplicate keys

Out of scope for CA1 (planned for later phases)

- Saving records to a file (CA2)
- Update and Delete (basic update in CA2, safe update and delete in CA3)
- Donation types such as platelet or plasma donors (CA3)
- Medical checks done at the camp (haemoglobin, blood pressure, medical history). The system records only details a donor can declare; the final decision is always the medical officer's.

4. Entities and unique key

The system has one main entity, **Donor**, identified by a unique **Donor ID** in the format `D` followed by 4 digits (for example `D1001`).

Entity	Role in CA1	Key
Donor (class)	One registered blood donor and their last donation	Donor ID, e.g. D1001
Date (struct)	A calendar date used for the last donation and today's	Not a stored entity

Entity	Role in CA1	Key
	date	

Why Donor ID and not the phone number? A phone number can change, and family members often share one number. A Donor ID never changes once issued, so it is a safer unique key. The system rejects a new donor whose ID already exists, and IDs are matched case-insensitively (`d1001` is treated as `D1001`).

Values that are calculated, not stored: the next eligible date, the number of days left, and the status text (Eligible now / Due in N days / Wait N days). They depend on today's date, so storing them would make them wrong the next day.

5. Main class design

The `Donor` class keeps all ten data members private and exposes only the functions below; the eligibility rule values are public static constants so every part of the program uses the same numbers.

Member function	Kind	What it does
`setDetails(...)`	Public	Stores all details in the object; uses `this->` because parameter names match member names
`getId()`, `getName()`, `getBloodGroup()`	Inline (defined inside class)	Return private values for searching and display
`isFirstTimeDonor()`	Inline (keyword `inline`)	True if the donor has never donated
`getGapDays()`	Inline (keyword `inline`)	Returns 90 for male, 120 for female donors
`hasSameId(id)`	Public	Compares `this->donorId` with an ID; used for duplicate checking and search
`getNextEligibleDate()`	Public	Last donation date plus the gap
`display(today)`	Public	Prints the full record of one donor
`displayRow(today)`	Public	Prints one row of the donor table
`getDonorCount()`, `increaseDonorCount()`	Static	Read and update the shared count of registered donors
`statusText(daysLeft)`	Static	Converts days left into "Eligible now", "Due in N days" or "Wait N days"
`daysUntilEligible(donor, today)`	Friend	Reads private `hasDonatedBefore` and `lastDonation`; 0 or less means eligible today

Static constants (shared rule values): MIN_AGE = 18, MAX_AGE = 65, MAX_FIRST_TIME_AGE = 60, MIN_WEIGHT = 45, MAX_WEIGHT = 200, MALE_GAP_DAYS = 90, FEMALE_GAP_DAYS = 120, REMINDER_DAYS = 7.

Program files

File	Contents
src/Donor.h, src/Donor.cpp	Donor class, static members, friend function
src/Utils.h, src/Utils.cpp	Date struct, date maths, string helpers, validators, safe input functions
src/main.cpp	Array of donors, menu, add / show / search / reminder operations

6. Data dictionary

Every stored field of a donor, its C++ type and the rule it must pass before it is saved.

Field	C++ type	Size / range	Validation rule	Example
donorId	std::string	5 characters	D + 4 digits; must not already exist; input converted to uppercase	D1001
name	std::string	2–30 characters	Letters, spaces and dots only; must start with a letter	Rahul Sharma
gender	char	1 character	M or F (lowercase accepted)	M
age	int	18–65 years	Whole number; a first-time donor must be 60 or younger	25
hasDonatedBefore	bool	true / false	Entered as Y or N	true
lastDonation	Date (struct)	DD-MM-YYYY	Real calendar date (leap years checked); not in the future; asked only if the donor has donated before	01-08-2026
weight	int	45–200 kg	Whole number; 45 kg is the NBTC minimum	68
bloodGroup	std::string	2–3 characters	One of A+, A-, B+, B-, AB+, AB-, O+, O-; input converted to	B+

Field	C++ type	Size / range	Validation rule	Example
			uppercase	
phone	std::string	10 digits	Digits only; first digit 6, 7, 8 or 9 (Indian mobile)	9876543210
city	std::string	2–25 characters	Letters and spaces; must start with a letter	Phagwara
donorCount	static int	0–100	Shared by the class; registration stops at MAX_DONORS (100)	4

Date struct: day (int, 1–31), month (int, 1–12), year (int, 1900–2100).

Known limitation: age is recorded at registration and does not increase on its own. Storing a date of birth instead is planned for CA2, when records are saved to a file and kept for longer.

7. UML class diagram

The main program owns an array of up to 100 Donor objects, each Donor contains one Date, and both use the Utils helper functions.

Notation: - private, + public, underlined members are static, a filled diamond is composition (the part cannot exist without its owner), and a dashed arrow is a «uses» dependency.

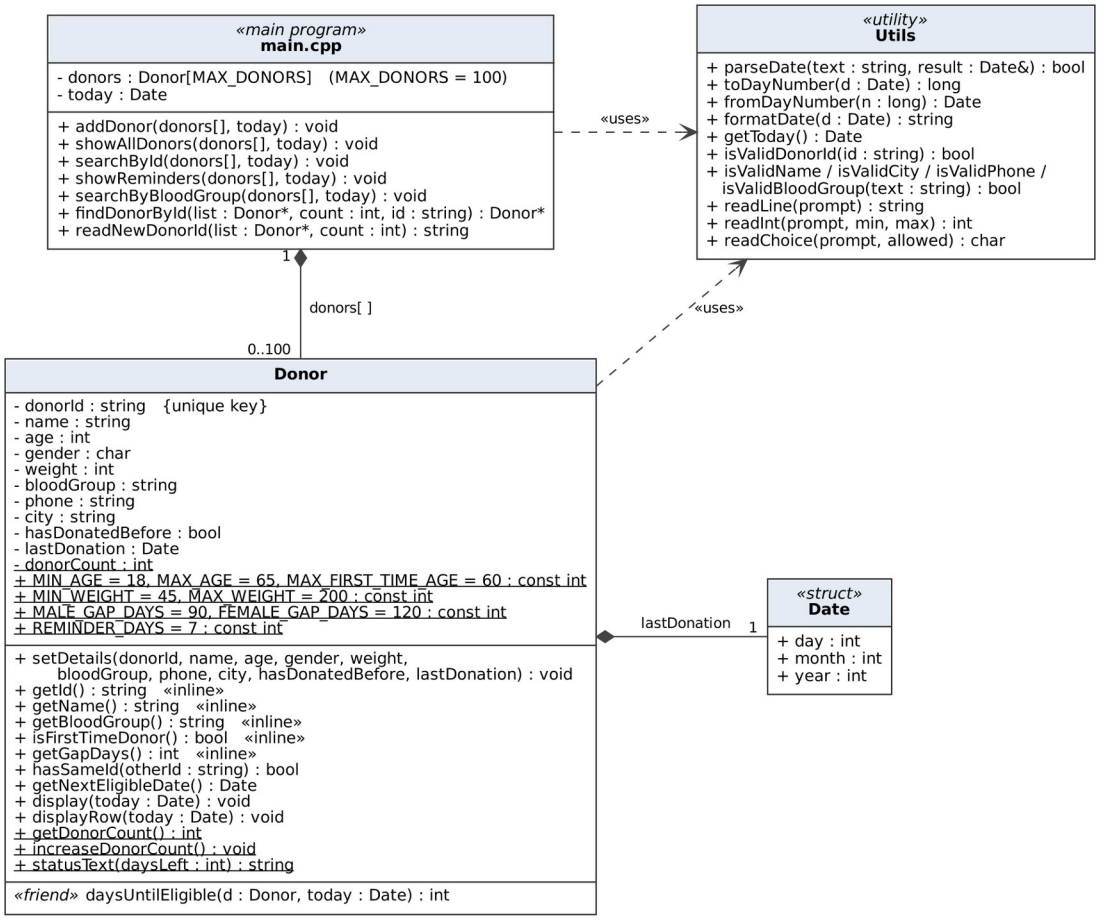


Figure 1: UML class diagram (source: `docs/uml_class_diagram.dot`)

8. Flowcharts

The program loops on a menu until the user chooses 0; adding a donor is the only operation that changes data, so it has its own flowchart showing every validation branch.

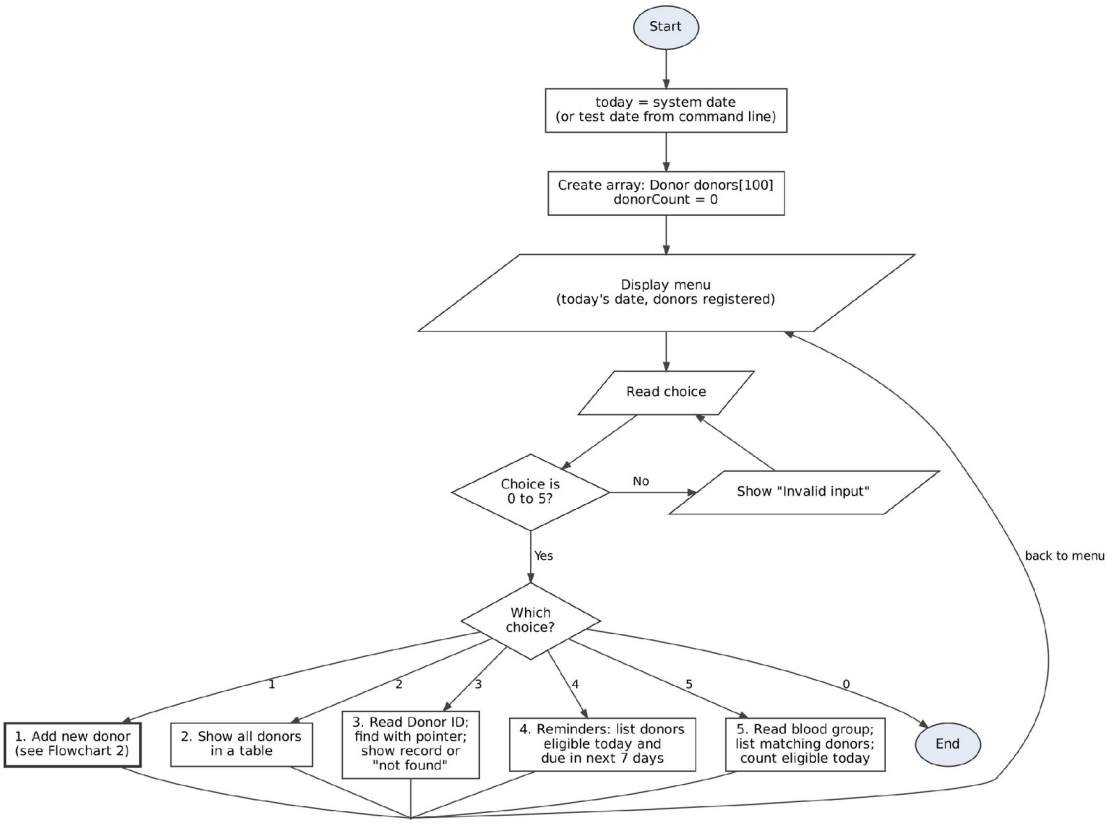


Figure 2: Main menu flow (source: docs/flowchart_main_menu.dot)

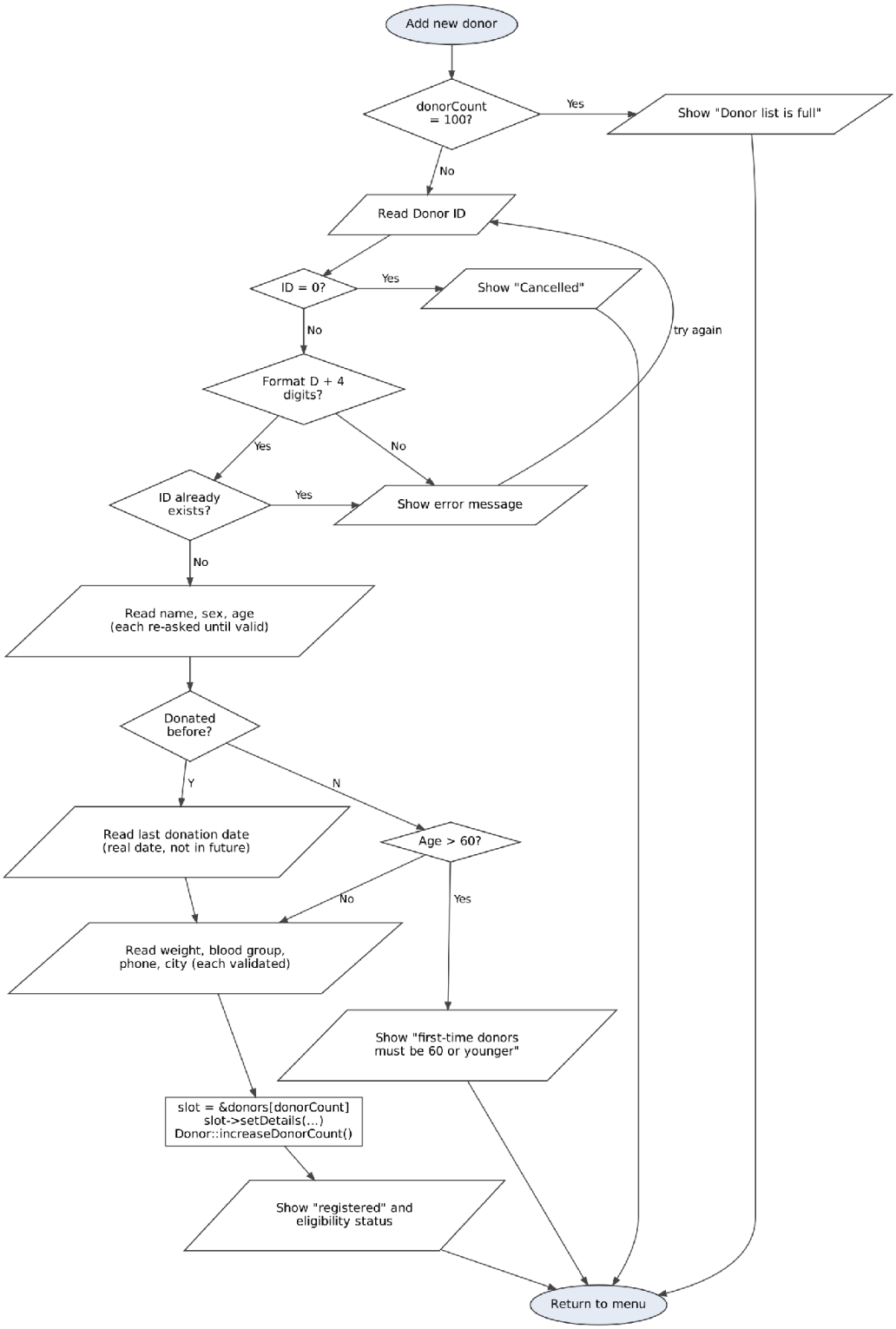


Figure 3: Add new donor, the Create operation with validation and duplicate-key check (source: docs/flowchart_add_donor.dot)

9. Eligibility rules used

The program applies the NBTC 2017 donor-selection rules that can be checked from details a donor declares; checks that need a medical examination are left to the blood bank.

Rule (NBTC 2017)	Value	How the program applies it
Age	18–65 years	Age input accepts only 18 to 65
First-time donor age	Not more than 60 years	A first-time donor older than 60 is not registered
Minimum weight	45 kg (350 ml collection)	Weight input accepts only 45 kg and above
Gap between whole blood donations, male	90 days	`getGapDays()` returns 90 for `M`
Gap between whole blood donations, female	120 days	`getGapDays()` returns 120 for `F`
Haemoglobin	At least 12.5 g/dL	Not checked: needs a test at the camp

How eligibility is calculated

1. Convert the last donation date to a day number (days counted from 01-01-1900).
2. Add the gap (90 or 120 days) to get the next eligible day, and convert it back to a date for display.
3. Days left = next eligible day − today's day number.
4. Days left $\leq 0 \rightarrow$ **Eligible now**; 1 to 7 $\rightarrow$ **Due in N days** (shown in the reminder list); more than 7 $\rightarrow$ **Wait N days**.
5. A first-time donor has no waiting period and is always **Eligible now**.

Counting days from a fixed starting date makes month lengths and leap years simple: the program only adds and subtracts whole numbers.

Sources: Revised NBTC Guidelines for Blood Donor Selection and Blood Donor Deferral, June 2017 (Dr Akshaya Tomar, AFMC Pune) · Blood transfusion: Guidelines by MoHFW, M3 India

10. Validation behaviour

No invalid value is ever stored: each field is re-asked until it is valid, with a message that says exactly what is expected.

- **All input is read with `getline`**, never `cin >>`. A stray newline can then never skip the next prompt, and typing letters where a number is expected cannot break the program.
- **Duplicate check comes first.** The Donor ID is checked against existing donors before any other field is asked, so the user does not type a full record only to have it rejected.
- **Cancel option.** Entering `0` as the Donor ID cancels the registration.
- **Rule check during entry.** If a first-time donor is older than 60, registration stops right after the "donated before?" question.

- **Full list.** When 100 donors are registered, Add is refused with a message instead of writing past the end of the array.
- **End of input.** If input ends unexpectedly (for example a test file runs out), the program exits cleanly instead of looping forever.

11. CA1 requirements and where they are in the code

Every CA1 item in the project guide is used for a real purpose in the program, not added only to tick a box.

CA1 requirement	Where	How it is used
Entity class	Donor in Donor.h	10 private data members, public member functions
Static members	donorCount, getDonorCount(), increaseDonorCount(), statusText(), rule constants	One shared donor count for the whole class; rule values defined once
Inline functions	Getters inside the class; isFirstTimeDonor(), getGapDays() with inline	Small functions called often while displaying tables
Friend function	daysUntilEligible()	Reads private hasDonatedBefore and lastDonation to compute days left
Array of objects	Donor donors[MAX_DONORS] in main()	In-memory storage for up to 100 donors
Pointers	findDonorById(), Donor* slot, for (Donor* p = donors; ...) loops	Search returns the address of the match or nullptr; loops walk the array with pointer arithmetic
this pointer	setDetails(), hasSameId(), display()	Separates members from same-named parameters; passes the current object to the friend function
std::string	All text fields and helpers	substr, length, find, find_first_not_of; uppercase conversion
Create with validation	addDonor() and the read...() functions	Every field re-asked until valid
Read	Show all, search by ID, reminders, find by blood group	Four read views over the same array
No duplicate keys	readNewDonorId()	Calls findDonorById() before accepting an ID

Friend function (Donor.cpp)

```
int daysUntilEligible(const Donor& d, const Date& today) {
    if (!d.hasDonatedBefore) {
        return 0;   // first-time donors have no waiting period
    }
    long nextDay = toDayNumber(d.lastDonation) + d.getGapDays();
    return (int)(nextDay - toDayNumber(today));
}
```

Search with a pointer (main.cpp)

```
Donor* findDonorById(Donor* list, int count, const string& id) {
    for (Donor* p = list; p < list + count; p++) {
        if (p->hasSameId(id)) {
            return p;
        }
    }
    return nullptr;
}
```

12. Test cases and results

All 33 console test cases (8 normal, 18 invalid, 7 boundary) and all 27 automated checks passed. Every console run used the fixed test date 05-11-2026 (`donor_app 05-11-2026`), so the results are the same on any day; full transcripts are in the `tests` folder.

Normal cases (tests/test1_normal_output.txt)

ID	Input	Expected	Actual	Result
N1	Add D1001, male, last donation 01-08-2026	Registered; next eligible 30-10-2026; Eligible now	Same	Pass
N2	Add D1002, female, last donation 10-07-2026	Next eligible 07-11-2026; Due in 2 days	Same	Pass
N3	Add D1003, male, last donation 01-10-2026	Next eligible 30-12-2026; Wait 55 days	Same	Pass
N4	Add D1004, first-time donor	Last donated "Never"; Eligible now	Same	Pass
N5	Menu 2: show all	Table of 4 donors	Same	Pass
N6	Menu 3: search D1002	Full record of Priya Verma	Same	Pass
N7	Menu 4: reminders	2 eligible today (D1001, D1004); 1 due soon (D1002)	Same	Pass
N8	Menu 5: blood group B+	2 donors listed; "1 of 2 can donate today"	Same	Pass

Invalid cases (tests/test2_invalid_output.txt)

ID	Input	Expected	Actual	Result
I1	Menu choice 9, then x	Error; asked again	Same	Pass

ID	Input	Expected	Actual	Result
I2	Search by ID with no donors	"No donors registered yet"	Same	Pass
I3	Donor ID 1001, then D10	Format error each time	Same	Pass
I4	Donor ID d1001 when D1001 exists	Duplicate rejected	Same	Pass
I5	Name A, then R@hul	Rejected	Same	Pass
I6	Sex X	Rejected	Same	Pass
I7	Age 17, 66, abc	Rejected	Same	Pass
I8	"Donated before?" maybe	Rejected	Same	Pass
I9	Date 2026-08-01, then 31-02-2026	Invalid date	Same	Pass
I10	Date 10-12-2026 (future)	"Cannot be in the future"	Same	Pass
I11	Weight 44, then 201	Rejected	Same	Pass
I12	Blood group C+, then ab+	C+ rejected; ab+ stored as AB+	Same	Pass
I13	Phone 12345, 5876543210, 98765abc10	Rejected	Same	Pass
I14	City Ph4gwara	Rejected	Same	Pass
I15	Donor ID 0	Cancelled; no donor added	Same	Pass
I16	First-time donor aged 61	Not registered	Same	Pass
I17	Search ID D9999	"No donor found"	Same	Pass
I18	Blood group A+ with no A+ donors	"No donors with blood group A+"	Same	Pass

Boundary cases (tests/test3_boundary_output.txt)

ID	Input	Expected	Actual	Result
B1	Male, age 65, weight 45, donated exactly 90 days ago	Accepted; Eligible now	Same	Pass
B2	Female, age 18, weight 200, donated 119 days ago	Accepted; Due in 1 day	Same	Pass
B3	First-time donor aged 60	Accepted; Eligible now	Same	Pass
B4	Date 29-02-2026, then 29-02-2024	2026 rejected (not a leap year); 2024 accepted	Same	Pass
B5	Male, 7 days left	In reminder list: Due in 7 days	Same	Pass
B6	Male, 8 days left	Not in reminder list: Wait 8 days	Same	Pass

ID	Input	Expected	Actual	Result
B7	Last donation = today (05-11-2026), female	Accepted; next eligible 05-03-2027; Wait 120 days	Same	Pass

Automated checks (tests/test_validators.cpp): 27 checks on the validators, date maths (including a year change) and the friend function. Result: 27 passed, 0 failed.

Sample output: reminder list (test N7)

```
Enter choice: 4

--- Eligibility Reminders for 05-11-2026 ---

Eligible to donate today: 2
ID     Name                Age  Sex Group  Phone       Last donated  Next eligible Status
-----------------------------------------------------------------------------------------------
D1001  Rahul Sharma        25   M   B+     9876543210  01-08-2026    30-10-2026    Eligible now
D1004  Simran Kaur         19   F   AB-    9123456780  Never         Any time      Eligible now

Becoming eligible in the next 7 days: 1
ID     Name                Age  Sex Group  Phone       Last donated  Next eligible Status
-----------------------------------------------------------------------------------------------
D1002  Priya Verma         22   F   O+     8765432109  10-07-2026    07-11-2026    Due in 2 days
```

Rendered from tests/test1_normal_output.txt.

13. Plan for later phases

Each phase extends the CA1 design without rewriting it; the one planned structural change is a fixed-size record for file storage in CA2.

Phase	Due (from allocation)	What will be added to this project
CA2 · Persist & Search	Day 14	Default, parameterised and copy constructors and a destructor for Donor; records saved to data/donors.dat as fixed-size binary records; search with seekg; basic update with seekp; data survives a restart; date of birth replaces age
CA3 · Reuse & Delete	Day 28	Base class Donor with derived classes for whole blood and apheresis (platelet/plasma) donors, each overriding a virtual getGapDays() (apheresis gap: 48 hours under NBTC 2017); overloaded <<, >>, ==, < and one more operator; a type conversion; new/delete with a virtual destructor; safe update and delete with confirmation
CA4 · Integrate & Defend	Day 42	Full CRUD menu; custom exception class with try/catch/throw; function and

Phase	Due (from allocation)	What will be added to this project
		class templates; std::vector with sort (by next eligible date), find_if and count_if; advanced feature: CSV export of the reminder list for camp volunteers, plus backup and restore

Design decision for CA2: a std::string member stores its text elsewhere in memory, so writing a Donor object straight to a binary file with write() would save memory addresses, not names. After a restart those addresses are meaningless. CA2 will therefore copy each donor into a separate fixed-size record struct (character arrays of known length) before writing, and back again after reading. Keeping Donor and the file record separate means the CA1 class does not need to change.

14. Repository layout, build and run

The repository follows the /docs, /src, /data, /tests layout from the project guide, with a README at the top level. Submission files are named 12604325_CAP455_<Phase>_<Title>, for example 12604325_CAP455_CA1_BloodDonorReminder.zip. Testing covers normal, invalid and boundary cases, including strict rejection of duplicate Donor IDs.

Repository link: [paste your GitHub link here]

I. /docs (documentation): This proposal (Word and PDF), the UML class diagram and the two flowcharts, saved as PNG images with their Graphviz .dot source files. The edge-case test table (CA3) and the final project report (CA4) will be added here in later phases.

II. /src (source code): main.cpp (menu and operations), Donor.h and Donor.cpp (the Donor class), and Utils.h and Utils.cpp (dates, validators and safe input). The code compiles on any standard C++11-or-later compiler; it was compiled with g++ 13 using -Wall -Wextra with no warnings.

III. /data (data storage): In CA1 donor records are kept in memory, so this folder holds a sample data file, sample_donors_input.txt, which registers six sample donors when it is given to the program as input. From CA2, when file persistence with fstream is added, the binary file donors.dat that saves the donor records will be stored here.

IV. /tests (testing): Input files and output transcripts for the normal, invalid and boundary test runs, and test_validators.cpp, which runs 27 automated checks on the validators, date calculations and eligibility rule.

V. README.md: Build and run steps and the features completed in each phase.

Build (g++ / MinGW, from the project folder)

```
g++ -std=c++17 -Wall src/main.cpp src/Donor.cpp src/Utils.cpp -o donor_app
```

Run

```
donor_app              uses today's date
donor_app 05-11-2026   test mode: treats 05-11-2026 as today
```

Code::Blocks: create a new Console Application project and add all five files from src, then Build and Run.

Re-run a test: donor_app 05-11-2026 < tests/test1_normal_input.txt

Load the sample donors: `donor_app < data/sample_donors_input.txt`

Automated checks: `g++ -std=c++17 tests/test_validators.cpp src/Donor.cpp src/Utils.cpp -Isrc -o run_tests`, then run `run_tests`.

15. References

1. National Blood Transfusion Council (NBTC), MoHFW. *Guidelines for Blood Donor Selection & Blood Donor Referral*, 2017, as summarised in: Tomar, A. Revised NBTC Guidelines for Blood Donor Selection and Blood Donor Deferral, Dept. of Immunohematology and Blood Transfusion, AFMC Pune.
2. M3 India Newsdesk. Blood transfusion: Guidelines by MoHFW, 14 June 2022.
3. Lovely Professional University. *CAP455 Minor Project Quick Guide*, Session 2026–27.
4. cppreference.com: std::basic_string — reference for the string functions used.